EvoCrypt: A Runtime-Upgradable Cryptographic Coprocessor for the Ethereum Virtual Machine

Anonymous Authors

Abstract. Smart contracts frequently rely on cryptographic algorithms, yet implementing complex cryptographic computation in Solidity expands it into large sequences of EVM instructions, resulting in substantial execution overhead and gas consumption. Precompiled Contracts reduce these costs by exposing client-side routines at fixed addresses, but they provide no runtime mechanism for evolving an algorithm independently of client releases and coordinated network upgrades. We present EvoCrypt, an upgradeable cryptographic coprocessor framework that exposes a stable contract-facing interface while executing replaceable WebAssembly (WASM) modules in a controlled client-side runtime. EvoCrypt distributes version metadata on-chain and deterministically activates an installed module at a designated block height, allowing participating nodes to switch algorithm versions consistently. Each invocation follows a deterministic gas rule: normally a version-registered charge, or the gas component of the corresponding upgrade proposal, avoiding expansion of the algorithm into EVM instructions while retaining explicit accounting. We implement EvoCrypt in Geth and evaluate it on a 20-node private network. The experiments show that EvoCrypt improves the efficiency of contract-facing cryptographic upgrades and invocations compared with Solidity-based implementations, while retaining near-Precompiled-Contract execution costs for most benchmark algorithms. EvoCrypt also supports the deployment of complex cryptographic algorithms that are difficult to realize directly in the EVM ecosystem.

Keywords: blockchain · smart contracts · upgrade · WebAssembly

1 Introduction

As blockchain systems become programmable platforms, smart contracts increasingly depend on cryptographic algorithms for authentication, integrity, signatures, and privacy [1–4]. The long-term security of these applications also requires the ability to adopt new algorithms, including post-quantum alternatives, without disrupting contract-facing functionality [5–7]. Blockchain clients therefore need an extensible way to provide evolving cryptographic capabilities to smart contracts.

Ethereum provides two main mechanisms for executing cryptographic operations: Solidity contract implementations and Precompiled Contracts. Solidity implementations are flexible and deployable at the contract layer, but complex cryptographic computation expands into costly EVM execution; Precompiled

Contracts execute selected routines efficiently in client-side code, but their supported algorithms are fixed to the client and normally require coordinated client updates to change [3, 4, 8].

Together, these mechanisms expose a fundamental trade-off between execution efficiency and algorithm upgradability. Solidity keeps algorithms deployable without client changes but can impose substantial execution and gas overhead, whereas Precompiled Contracts provide efficient execution only for routines embedded in the client and are therefore difficult to evolve independently. The central challenge is to replace contract-facing cryptographic implementations without sacrificing efficient execution, deterministic behavior across nodes, or compatibility with the existing EVM interface.

EvoCrypt addresses this trade-off by separating the stable contract-facing interface from replaceable WASM implementations and by separating module preparation from module activation. The interface preserves how smart contracts invoke cryptographic operations, while portable WASM modules allow implementations to be replaced without embedding each algorithm in contract bytecode or rebuilding the client for every change. Asynchronous preparation moves verification, compilation, and instantiation away from the transaction execution path. Activation-Block-based selection then ensures that nodes switch versions according to a common chain-derived rule rather than local readiness time. The design combines algorithm-level accounting with deterministic, incrementally deployable upgrades.

The main contributions of this article are summarized as follows.

- First, we design an upgradeable cryptographic coprocessor that separates the stable contract-facing interface from replaceable WASM-based algorithm implementations, allowing complex cryptographic operations to execute outside the EVM while preserving contract compatibility.
- Second, we design an activation-block-based upgrade protocol in which upgrade proposals are published on-chain. Each proposal has three parts: algorithm information, a gas rule bound to that algorithm, and control information comprising the Activation Block and the hash used to verify the WASM bytecode. Clients derive the same proposal from this record and therefore execute the same module once the upgrade becomes active.
- Finally, we implement EvoCrypt in Geth and evaluate it on a 20-node private blockchain network. The results show that EvoCrypt reduces the implementation burden and calling cost of the evaluated cryptographic operations while supporting dynamic evolution of contract-facing algorithms.

2 Related Work

Research on cryptographic algorithm upgrades in blockchain systems has not yet resolved the combined problem of efficient invocation and practical implementation. Related approaches can be organized by the layer at which they support upgrades: protocol and client, contract, or dynamic runtime.

Blockchain upgrades at the protocol or client layer replace execution rules through a coordinated transition. Cardano switches protocol eras with a hard-fork combinator at designated heights, and work on updatable blockchains studies how old and new chains stay synchronized [9–11]. Soft forks encode selected policy changes in rules that can be adopted without replacing the protocol [12]. Phoenix and Polkadot are closest to EvoCrypt in supporting live replacement: Phoenix compiles upgrade code carried in transactions at runtime, while Polkadot replaces on-chain WASM runtime logic during block synchronization [13, 14]. Both nevertheless replace client or chain-runtime components rather than a contract-facing cryptographic algorithm. They do not specify how a contract names an algorithm, binds its interface, or switches versions without redeploying the caller. EvoCrypt narrows the replacement boundary to individual WASM modules behind a stable Solidity-facing interface. Consistency follows from an on-chain proposal whose control fields carry the module hash and Activation Block, and whose gas fields are bound to the individual algorithm.

Contract-level schemes update application logic without replacing the entire blockchain client, using deployable contracts, proxy indirection, or bytecode rewriting. EVMPatch patches deployed Ethereum smart contracts through bytecode rewriting and proxy-based deployment, while Solidity implementations offer application-level flexibility within the EVM [15, 4]. LEAGAN extends contract-level upgradeability through data separation, incremental hashing, revision control, and status-contract-based version management [16]. Precompiled Contracts provide a complementary client-side path for selected cryptographic routines through reserved EVM-facing addresses [8, 17, 18]. Contract-level implementations remain constrained by EVM execution cost and state or interface compatibility, whereas adding or replacing a Precompiled Contract normally requires changes to client and protocol rules.

Dynamic software-update schemes move replacement below the application layer by loading new components into a running host. KylinX supports runtime library replacement through dynamic mapping, Mvedsua combines dynamic updates with multi-version execution, PYLIVE changes Python code online, and distributed-system research addresses modular updates and version consistency [19–23]. WASM provides a portable artifact format and explicit host boundary for this style of replacement. In blockchain systems, existing WASM-based upgrades generally target broad runtime logic rather than a narrow EVM-facing cryptographic interface [24, 14].

3 EvoCrypt Design

EvoCrypt is a cryptographic coprocessor framework for dynamically upgrading contract-facing cryptographic algorithms. This section presents the system model, the upgrade protocol, and the invocation protocol that together define the design. We focus on the coordination, version-selection, and contract-facing execution rules of EvoCrypt, while the prototype implementation and experimental configuration are described in Section 5.

3.1 System Model

EvoCrypt separates algorithm development, chain state, contract execution, and module execution into four cooperating components: the Community, the Blockchain, the EVM, and the Cryptographic Coprocessor. The Community includes developers who implement cryptographic algorithms and proposers who submit algorithm upgrades. The Blockchain maintains the canonical chain and shared world state, including smart contracts and the Management Contract. The EVM processes smart-contract transactions, while the node-local Cryptographic Coprocessor prepares, activates, and executes replaceable cryptographic modules.

Two chain-derived paths connect these components. A proposer records an upgrade through the Management Contract, whereas a contract initiates an invocation through the EVM of a Blockchain node. The Management Contract stores each admitted upgrade proposal, including the gas rule bound to its algorithm, and exposes the contract-facing upgrade and invocation interfaces. The coprocessor uses the resulting chain state to prepare and execute the corresponding module. Figure 1 summarizes these objects and their relationships.

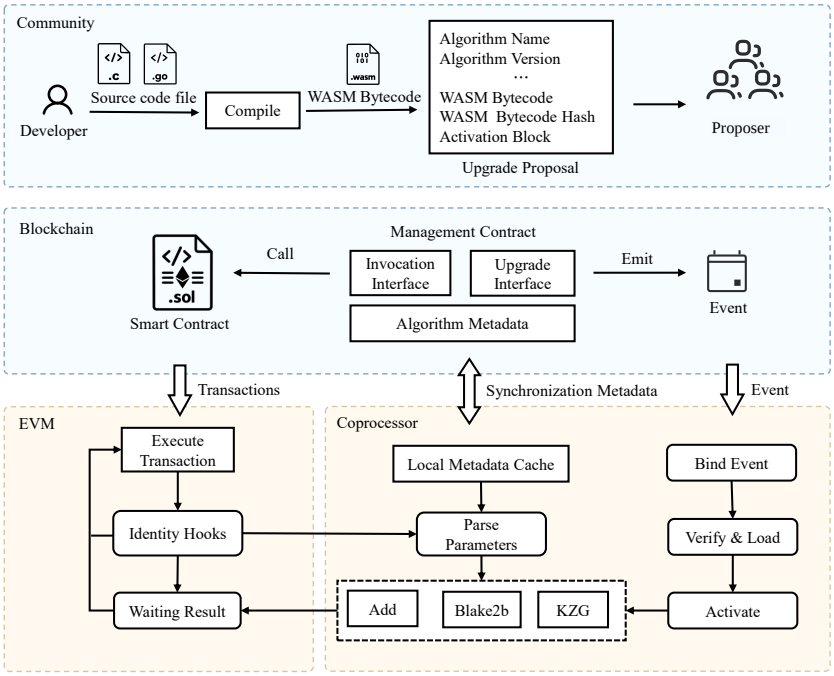


Fig. 1. System model of EvoCrypt.

3.2 Upgrade Protocol

On-chain publication. On-chain publication provides the authoritative record that coordinates node-local preparation and height-based activation. A proposer submits an upgrade proposal through the designated upgrade interface. The Management Contract records its algorithm, gas, and control components and emits an upgrade event.

The EVM records the proposal but does not compile or instantiate the WASM module. Node-dependent preparation therefore remains outside the transaction that changes the chain state. Every node reconstructs the same version from the admitted proposal. Its algorithm part names the algorithm and version and fixes the entry and output parameters; its gas part supplies the pricing rule bound to that algorithm; its control part supplies the Activation Block and the hash used to verify the bytecode. Only $\langle a, v, H_{a,v}\rangle$ is emitted as a notification index. Let $\mathcal{V}_a$ denote the versions already registered for algorithm a; the authoritative record prevents nodes from preparing different artifacts or execution rules for the same upgrade.

Definition 1 (Upgrade proposal). *Let a be an algorithm name and v a version number. An admitted upgrade proposal is*

$$\begin{aligned} \mu_{a,v} &= \big(\mathsf{Alg}_{a,v},\, \Gamma_{a,v},\, \mathsf{Ctl}_{a,v}\big), \\ \mathsf{Alg}_{a,v} &= \big(a,\, v,\, C_{a,v},\, \tau^{\text{in}}_{a,v},\, \tau^{\text{out}}_{a,v}\big), \\ \Gamma_{a,v} &= \big(g^{\text{reg}}_{a,v},\, B_a,\, \alpha_a,\, N_{0,a}\big), \\ \mathsf{Ctl}_{a,v} &= \big(H_{a,v},\, h_{a,v}\big), \end{aligned} \tag{1}$$

where $C_{a,v}$ is the WASM bytecode, $\tau^{\text{in}}_{a,v}$ and $\tau^{\text{out}}_{a,v}$ are its ABI descriptors, $g^{\text{reg}}_{a,v}$ is the registered charge, $(B_a, \alpha_a, N_{0,a})$ are the pricing parameters, $H_{a,v}$ is the Activation Block, and $h_{a,v} = \text{Keccak256}(C_{a,v})$.

Node-local preparation. Node-local preparation converts the admitted metadata record into executable state without changing the active version. After observing the upgrade event, each client recovers $\mu_{a,v}$, checks that the event index agrees with the stored record, decodes the artifact, and recomputes its hash. The Upgrade Handler then compiles and instantiates the module and validates its required entry point. Once these checks succeed, the node records (a, v) as prepared. Algorithm 1 gives the complete publication and preparation procedure.

Preparation is deliberately separated from the chain-level upgrade record. Node-local work can proceed asynchronously, failed attempts can retry from the authoritative chain record, and an executed upgrade transaction is not rolled back. The separation allows preparation to complete without changing the version selected by the chain.

Height-based activation. Height-based activation makes the selected version a function of shared chain height rather than node-local preparation time. The Upgrade Block includes the admitted upgrade transaction and emits $\langle a, v, H_{a,v}\rangle$. After that block, each node independently completes the off-chain Load–Verify–Ready sequence, while invocations before $H_{a,v}$ continue to use the previously

Algorithm 1 Activation-Block-based WASM upgrade protocol.

Require: Upgrade proposal $\mu_{a,v}$ as in (1); node set $\mathcal{N}$
Ensure: Prepared set updated on every $n_i \in \mathcal{N}$

proposer submits $\mu_{a,v}$ through the designated upgrade interface
Management Contract stores $\mu_{a,v}$; emits event $\langle a, v, H_{a,v} \rangle$
for all $n_i \in \mathcal{N}$ **do**
    observe $\langle a, v, H_{a,v} \rangle$; fetch $\mu'_{a,v}$ from Management Contract
    if event fields disagree with $\mu'_{a,v}$ **then**
        reject (a, v)
    else
        decode $\widehat{C}$ from $\mu'_{a,v}$; reject (a, v) unless Keccak256$(\widehat{C}) = h_{a,v}$
        repeat
            compile and instantiate $\widehat{C}$; validate the required module entry point
        until preparation succeeds
        persist $\widehat{C}$ under $h_{a,v}$ with $\mu_{a,v}$; mark (a, v) prepared
    end if
end for

selected version. At the Activation Block, nodes with the same chain view select version v according to the rule below. Compilation-time differences therefore cannot change the version selected at a given execution height.

When a selected version is not yet prepared, the node reloads its authoritative metadata and continues preparation rather than selecting an obsolete version. The policy preserves height-based selection while allowing node-local readiness to complete asynchronously. Figure 2 shows the separation between publication, preparation, and activation.

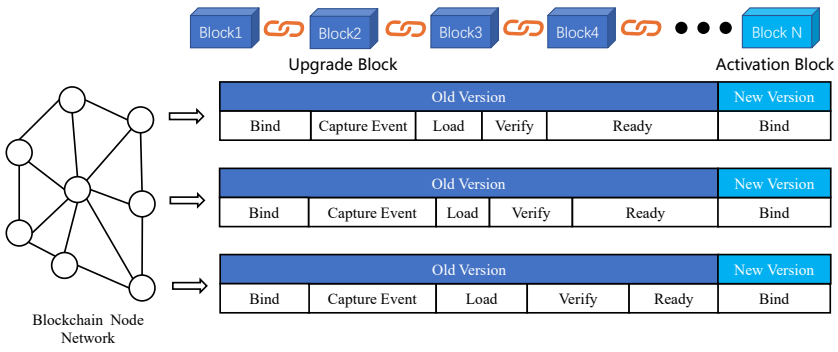


Fig. 2. Node-local preparation and deterministic activation.

Definition 2 (Active version). *Let $\mathcal{V}_a$ be the registered versions of algorithm a and b the execution height. The version used at height b is*

$$v^*(a, b) = \arg\max\{(H_{a,v}, v) : v \in \mathcal{V}_a,\ H_{a,v} \le b\}, \tag{2}$$

in lexicographic order, later activation height first and then larger version number. If no such version exists, the invocation fails with a missing-version error.

3.3 Invocation Protocol

A unified Management Contract entry point keeps the contract-facing call boundary stable. The caller serializes the algorithm name a and its parameters into an encoded request x and submits it through the designated interface, so different algorithms share one destination address and the contract binds to the ABI descriptors of the selected version rather than to module location, implementation language, or node-local loading state. EVM-side Hooks admit only requests to that interface; all others remain on the ordinary EVM path. For a routed request the coprocessor parses (a, x), resolves $v^*(a, b)$ by (2), and, under the mandatory-preparation policy, prepares the selected version if necessary rather than falling back to an obsolete implementation. The selected module receives the encoded parameters and returns an encoded result; the call shape remains compatible when $\tau^{\text{in}}_{a,v}$ and $\tau^{\text{out}}_{a,v}$ are compatible. Algorithm 2 specifies this procedure and its protocol-error conditions.

Definition 3 (Algorithm-level gas schedule). *Let $i \geq 1$ be one plus the number of earlier charged calls to algorithm a in the current transaction, resetting for each transaction and algorithm, and let b be the execution height. Using the pricing parameters in $\Gamma_{a,v}$, the charge $G(a, v, x, i, b)$ equals $G_{\text{prof}}(a, v, x, i)$ when $\Gamma_{a,v}$ supplies a pricing rule, and equals $g^{\text{reg}}_{a,v}$ otherwise:*

$$G_{\text{prof}}(a, v, x, i) = \left\lceil B_a(|x|)(1 + \alpha_a)^{\max(0, i - N_{0,a})} \right\rceil . \tag{3}$$

G_{prof} replaces $g^{\text{reg}}_{a,v}$ for that call. An invalid or missing configured version is an error.

Because G depends only on the selected version, the pricing parameters in the admitted proposal, the encoded input size, and the transaction-local call index, nodes executing the same transaction prefix charge the same amount independently of local CPU time or WASM instruction counts. The coefficients used for the evaluated algorithm in Section 5 demonstrate an experimental tariff rather than a calibrated economic optimum.

4 Security Analysis

This section formalizes the upgrade protocol and derives the invocation guarantees from it. Section 4.1 fixes the ledger and registry abstractions, the trusted assumptions, the adversary, and game-based definitions for the two failure modes an upgrade mechanism must exclude: honest nodes selecting different versions for the same height, and an honest node executing bytecode other than the admitted artifact. Section 4.2 shows that EvoCrypt excludes the first unconditionally relative to the ledger abstraction and the second by a tight reduction

Algorithm 2 Contract-to-coprocessor invocation protocol.

Require: Recognized invocation request d, block height b, gas budget g
Ensure: ABI-encoded result y or explicit error

$(a, x) \leftarrow \text{Parse}(d)$
if the request is invalid **then**
    return ABI error
end if
$v \leftarrow v^*(a, b)$ by (2)
if no applicable version exists **then**
    return missing-version error
end if
ensure (a, v) is prepared under the mandatory-preparation policy
$G_{\text{req}} \leftarrow G(a, v, x, i, b)$ by Definition 3
if $g < G_{\text{req}}$ **then**
    return out-of-gas error
end if
charge G_{req}; $y \leftarrow \text{Execute}(a, v, x)$
return y, or runtime error on failure

to the collision resistance of Keccak256, and combines both into an upgrade-soundness theorem. Section 4.3 extends consistency to invocation outcomes and proves resistance to gas-charge evasion.

4.1 Security Model

Notation and ledger. κ is the security parameter, $\mathsf{negl}(\kappa)$ a negligible function, and algorithms are probabilistic polynomial time (PPT) unless stated otherwise. $\mathcal{N}$ is the set of honest nodes, each running Algorithms 1 and 2. The proposal $\mu_{a,v}$ and the fields $C_{a,v}$, $h_{a,v}$, and $H_{a,v}$ are the algorithm and control components in Definition 1, and $u_{a,v}$ denotes the height of the Upgrade Block that admitted $\mu_{a,v}$. We abstract the blockchain as a transaction ledger: honest node n holds a local view $\mathcal{L}_n$, a finite block sequence with prefix $\mathcal{L}_n[1..b]$ and height $|\mathcal{L}_n|$. Transaction order inside a block is part of the ledger, so an execution point is a height together with a position in the block; we write b for both.

Definition 4 (Registry state). *The registry* $\mathsf{Reg}(P)$ *of a ledger prefix* P *is the Management Contract storage obtained by executing* P *from the genesis state; by EVM determinism it is a function of* P*. For a node* n *and height* b*, let* $\mathcal{V}_a^n(b) = \{v \mid \mu_{a,v} \in \mathsf{Reg}(\mathcal{L}_n[1..b])\}$ *and let* $v_n^*(a, b)$ *be the value of* (2) *over* $\mathcal{V}_a^n(b)$*, with* $v_n^*(a, b) = \perp$ *when the feasible set is empty.*

Definition 4 fixes the version each honest node selects from its ledger prefix. Agreement on that version, and execution of the admitted artifact, both depend on assumptions that we now make explicit. Assumptions 1–3 are used for upgrade soundness in Section 4.2, and Assumption 4 only when Section 4.3 extends consistency to invocation outcomes.

Assumption 1 (Canonical ledger). *For all honest $n, n' \in \mathcal{N}$ and every $b \leq \min(|\mathcal{L}_n|, |\mathcal{L}_{n'}|)$, $\mathcal{L}_n[1..b] = \mathcal{L}_{n'}[1..b]$. The gas parameters on that prefix are those in the admitted proposals.*

Prefix equality expresses ledger persistence [25]; the gas parameters are fields of the admitted proposals on that prefix. Blocks subject to reorganization are re-executed by ordinary EVM rules and lie outside the model.

Assumption 2 (Registry validity). *Every upgrade record $\mu_{a,v}$ admitted by the Management Contract is well formed: (i) $h_{a,v} = \mathrm{Keccak256}(C_{a,v})$; (ii) $u_{a,v} < H_{a,v}$, i.e., the record is committed strictly before its Activation Block; (iii) (a, v) is unique, and an admitted record is never modified or removed; (iv) $C_{a,v}$ passes validation of the configured WASM Runtime. Conditions (i)–(iii) can be enforced by the contract itself; listing them as assumptions keeps the analysis independent of the contract implementation.*

Assumption 3 (Collision resistance). *For every PPT algorithm $\mathcal{B}$,*

$$\mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B}) = \Pr\big[(C, C') \leftarrow \mathcal{B}(1^\kappa) : C \neq C' \wedge \mathrm{Keccak256}(C) = \mathrm{Keccak256}(C')\big] \leq \mathsf{negl}(\kappa). \tag{4}$$

Keccak256 is unkeyed, so (4) is read in the concrete sense of Rogaway [26]: each reduction below exhibits an explicit collision finder $\mathcal{B}$ and bounds the protocol adversary's advantage by that of $\mathcal{B}$.

Assumption 4 (Deterministic runtime). *The WASM Runtime exposes a validation predicate $\mathsf{Validate}(C) \in \{0, 1\}$ and an execution function $\mathsf{Exec}(C, x)$ whose outputs, including error outcomes, depend only on (C, x) and the fixed, allowlisted host interface. It enforces bounded linear memory, import validation and the configured resource limits, and exposes no host state or capability beyond that interface.*

Adversary. The PPT adversary $\mathcal{A}$ may (a) submit arbitrary transactions, including invocation inputs and candidate upgrade proposals, the latter admitted only subject to Assumption 2; (b) schedule the delivery of upgrade events, delay or interrupt node-local preparation, and crash and restart nodes so that preparation retries; and (c) tamper with the node-local artifact path, i.e., replace the byte sequence $\widehat{C}$ that a node obtains when it decodes, persists or reloads an artifact, modelling corrupted caches, persisted copies and untrusted retrieval channels. It cannot violate Assumptions 1–3 or alter the client binary. The model thus excludes a compromised Geth binary, a broken consensus protocol, and a primitive whose underlying assumption is false. Registry validity does not imply that a registered implementation is cryptographically correct or that its gas parameters accurately price resource use.

Security definitions. Each experiment below runs $\mathcal{A}$ against the honest nodes in this model and outputs 1 when $\mathcal{A}$ causes the undesirable event; the advantage of $\mathcal{A}$ is the probability of output 1 over all coins.

Definition 5 (Version divergence). *In* $\mathsf{Exp}^{\mathrm{div}}_{\mathcal{A}}(\kappa)$*, the adversary finally outputs a tuple* (a, b, n, n') *with* $n, n' \in \mathcal{N}$ *and* $b \leq \min(|\mathcal{L}_n|, |\mathcal{L}_{n'}|)$*. The experiment outputs* 1 *iff* $v^*_n(a, b) \neq v^*_{n'}(a, b)$*, where* $\perp$ *differs from every version.* $\mathsf{Adv}^{\mathrm{div}}_{\mathcal{A}}(\kappa) = \Pr[\mathsf{Exp}^{\mathrm{div}}_{\mathcal{A}}(\kappa) = 1]$.

Definition 6 (Module substitution). $\mathsf{Exp}^{\mathrm{sub}}_{\mathcal{A}}(\kappa)$ *outputs* 1 *iff some honest node* n *marks* (a, v) *as prepared with, or executes for* (a, v)*, a byte sequence* $\widehat{C}$ *such that* $\mu_{a,v} \in \mathsf{Reg}(\mathcal{L}_n)$ *and* $\widehat{C} \neq C_{a,v}$. $\mathsf{Adv}^{\mathrm{sub}}_{\mathcal{A}}(\kappa) = \Pr[\mathsf{Exp}^{\mathrm{sub}}_{\mathcal{A}}(\kappa) = 1]$.

Definition 7 (Upgrade soundness). $\mathsf{Exp}^{\mathrm{bind}}_{\mathcal{A}}(\kappa)$ *outputs* 1 *iff some honest node executes an invocation of* a *at height* b *with a byte sequence* $\widehat{C} \neq C_{a,v^*(a,b)}$*, where* $v^*(a, b)$ *is evaluated on the canonical prefix of height* b*; executing any module when* $v^*(a, b) = \perp$ *also counts as output* 1*. EvoCrypt is* upgrade sound *if* $\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A}}(\kappa) = \Pr[\mathsf{Exp}^{\mathrm{bind}}_{\mathcal{A}}(\kappa) = 1]$ *is negligible for every PPT* $\mathcal{A}$*.*

Definition 7 is the target property: whatever $\mathcal{A}$ does to scheduling and node-local storage, every honest node that answers an invocation runs exactly the artifact the canonical chain designates for that height. Definitions 5 and 6 isolate the two ways this can fail.

Definition 8 (Gas-charge evasion). *In* $\mathsf{Exp}^{\mathrm{gas}}_{\mathcal{A}}(\kappa)$*,* $\mathcal{A}$ *submits invocation transactions, choosing their inputs, call-gas budgets, and repeated calls, and finally identifies an invocation of* a *at execution point* b *on an honest node* n*. Let* x *be its parsed input,* g *its available gas before charging, and* i *one plus the number of earlier charged calls to* a *in the same transaction, as in Definition 3. Set* $G^* = G(a, v^*(a, b), x, i, b)$ *using the gas component of the admitted proposal for* a*. The experiment outputs* 1 *iff the invocation reaches line 14 with either* $g < G^*$ *or a coprocessor charge* $\gamma < G^*$*; it outputs* 0 *if the request or pricing data is invalid, no version is active, or execution is not reached. Thus the attack succeeds by bypassing the gas-budget check or obtaining execution below the scheduled charge. Define* $\mathsf{Adv}^{\mathrm{gas}}_{\mathcal{A}}(\kappa) = \Pr[\mathsf{Exp}^{\mathrm{gas}}_{\mathcal{A}}(\kappa) = 1]$.

4.2 Upgrade Security

Lemma 1 (Activation timeliness). *Under Assumption 2, for every honest node* n *and every* $\mu_{a,v} \in \mathsf{Reg}(\mathcal{L}_n)$*: (i)* $v^*_n(a, b) \neq v$ *for every* $b < H_{a,v}$*, and (ii)* v *lies in the feasible set of* (2) *at every* b *with* $H_{a,v} \leq b \leq |\mathcal{L}_n|$*.*

Proof. (i) The feasible set contains only versions with $H_{a,v} \leq b$. (ii) By Assumption 2(ii), $u_{a,v} < H_{a,v} \leq b$, so the admitting transaction lies in $\mathcal{L}_n[1..b]$, and by (iii) the record is never removed; hence $\mu_{a,v} \in \mathsf{Reg}(\mathcal{L}_n[1..b])$ and $H_{a,v} \leq b$ places v in the feasible set. □

The lemma confines the preparation window of (a, v) to heights $u_{a,v} < b < H_{a,v}$ and excludes retroactive activation: appending a block never changes the version designated for an already-executed height, so a node replaying history agrees with nodes that executed it live.

Theorem 1 (Activation consistency). *Under Assumptions 1 and 2, every adversary $\mathcal{A}$, including a computationally unbounded one, has* $\mathsf{Adv}^{\mathrm{div}}_{\mathcal{A}}(\kappa) = 0$.

Proof. Fix any output (a, b, n, n') of $\mathcal{A}$. By Assumption 1, $P = \mathcal{L}_n[1..b] = \mathcal{L}_{n'}[1..b]$, and by Definition 4 $\mathsf{Reg}(P)$ is a function of P, so both nodes hold the same records and the same feasible set $\{(H_{a,v}, v) : H_{a,v} \le b\}$. By Assumption 2(iii) its elements are pairwise distinct, so the lexicographic maximum in (2) is unique when the set is non-empty, and both nodes return $\perp$ when it is empty; hence $v^*_n(a, b) = v^*_{n'}(a, b)$. No adversarial capability enters this computation: scheduling, preparation timing, crashes and artifact tampering act on node-local state, whereas line 5 of Algorithm 2 evaluates (2) on the registry before any node-local state is consulted. The experiment therefore never outputs 1, whatever the running time of $\mathcal{A}$. □

Theorem 1 is information-theoretic. The next theorem shows that binding the executed bytecode to the registry costs exactly one collision-resistance assumption.

Theorem 2 (Module binding). *Under Assumption 2, for every PPT adversary $\mathcal{A}$ there exists a PPT algorithm $\mathcal{B}$, running in the time of $\mathcal{A}$ plus the cost of simulating the honest nodes, such that*

$$\mathsf{Adv}^{\mathrm{sub}}_{\mathcal{A}}(\kappa) \le \mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B}). \tag{5}$$

In particular, $\mathsf{Adv}^{\mathrm{sub}}_{\mathcal{A}}(\kappa)$ *is negligible under Assumption 3.*

Proof. On input 1^κ, $\mathcal{B}$ runs $\mathsf{Exp}^{\mathrm{sub}}_{\mathcal{A}}(\kappa)$ internally: it simulates the ledger and every honest node, executing Algorithms 1 and 2 on their behalf, and answers the scheduling and artifact-tampering requests of $\mathcal{A}$ exactly as the real experiment would. The simulation is perfect because the model contains no secret state. Whenever a simulated honest node is about to mark (a, v) prepared with $\widehat{C}$ (Algorithm 1, line 12) or to execute $\widehat{C}$ for (a, v) (Algorithm 2, line 14), $\mathcal{B}$ reads $C_{a,v}$ from the simulated registry; if $\widehat{C} \ne C_{a,v}$ it outputs $(\widehat{C}, C_{a,v})$ and halts, and otherwise it eventually outputs $\perp$.

Let E be the event that $\mathsf{Exp}^{\mathrm{sub}}_{\mathcal{A}}(\kappa)$ outputs 1; by perfect simulation, $\Pr[E] = \mathsf{Adv}^{\mathrm{sub}}_{\mathcal{A}}(\kappa)$ in the run of $\mathcal{B}$. On E, an honest node accepted $\widehat{C}$ for (a, v), which happens only after verifying $\mathrm{Keccak256}(\widehat{C}) = h_{a,v}$ against the canonical record (Algorithm 1, line 8; the prepared set is keyed by $h_{a,v}$, so execution reloads the artifact under the same check). By Assumption 2(i), $h_{a,v} = \mathrm{Keccak256}(C_{a,v})$, so $(\widehat{C}, C_{a,v})$ is a collision and $\Pr[E] \le \mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B})$, which is (5). The reduction is tight: it loses no advantage and does not guess the target record. □

The control hash binds only the bytecode. The entry and output descriptors, the gas component, and the Activation Block are the remaining fields of the same admitted proposal and are read from the canonical registry. These values are common to honest nodes under the stated ledger assumptions.

Theorem 3 (Upgrade soundness). *Under Assumptions 1–3, EvoCrypt is upgrade sound: for every PPT adversary $\mathcal{A}$ there exists a PPT $\mathcal{B}$ as in Theorem 2 such that*

$$\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A}}(\kappa) \le \mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B}). \tag{6}$$

Proof. Let W be the event that $\mathsf{Exp}^{\mathrm{bind}}_{\mathcal{A}}(\kappa)$ outputs 1 at honest node n for an invocation of a at height b executed with $\widehat{C}$; let $v = v^*(a,b)$ be the canonical selection, v_n the version resolved by n in line 5 of Algorithm 2, and $\hat{v}$ the version under which $\widehat{C}$ was prepared. Then $W \subseteq E_1 \cup E_2 \cup E_3$ with

$$E_1 : v_n \neq v, \qquad E_2 : v_n = v \wedge \hat{v} \neq v_n, \qquad E_3 : \hat{v} = v_n = v \wedge \widehat{C} \neq C_{a,v},$$

since if none occurs then $\widehat{C} = C_{a,v^*(a,b)}$. $\Pr[E_1] = 0$: node n evaluates (2) on $\mathsf{Reg}(\mathcal{L}_n[1..b])$, which by Assumption 1 is the canonical prefix, so the argument of Theorem 1 gives $v_n = v$; if $v = \perp$ the node returns a missing-version error (line 7) and executes nothing. $\Pr[E_2] = 0$: line 9 admits execution only for the resolved pair (a, v_n) and, under the mandatory-preparation policy, continues preparing (a, v_n) rather than falling back to another prepared version, so the only artifact reaching line 14 was prepared for (a, v_n). E_3 is a module substitution for (a, v) with $\mu_{a,v} \in \mathsf{Reg}(\mathcal{L}_n)$, the latter by Lemma 1(ii) since $H_{a,v} \le b$; hence $\Pr[E_3] \le \mathsf{Adv}^{\mathrm{sub}}_{\mathcal{A}}(\kappa) \le \mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B})$ by Theorem 2. A union bound yields (6). □

Remark 1 (Safety versus liveness). Theorems 1–3 are safety statements: an honest node never selects or runs a version other than the canonical one. They do not assert that every node can answer at height $H_{a,v}$; a node still preparing, or in the retry loop of Algorithm 1, defers rather than degrades. Under Assumption 2(iv) and the deterministic validation of Assumption 4, an admitted artifact validates identically at every honest node, so preparation cannot succeed at one node and fail permanently at another; preparation time is measured in Section 5.2 rather than proven here.

4.3 Invocation Security

Section 4.2 fixes *which* artifact an honest node executes. This subsection shows that, once Assumption 4 is added, agreement on the artifact lifts to agreement on the invocation outcome and on the resulting EVM state transition. Assumption 4 is the only new ingredient; it is an engineering assumption on the configured WASM Runtime rather than a cryptographic one, and the statements below make its role explicit. An *outcome* of Algorithm 2 is the pair (o, γ), where o is either the ABI-encoded result y or one of the protocol errors returned by the algorithm, and γ is the gas charged, with $\gamma = 0$ when the algorithm returns before line 14.

Definition 9 (Invocation divergence). *In $\mathsf{Exp}^{\mathrm{inv}}_{\mathcal{A}}(\kappa)$, the adversary outputs (d, g, b, i, n, n'), where $n, n' \in \mathcal{N}$ and $b \le \min(|\mathcal{L}_n|, |\mathcal{L}_{n'}|)$. Both nodes run Algorithm 2 with the same request, gas budget, height, and call index. For an ac-*

tual transaction, preceding charged calls determine i. If either node has not produced an outcome, the experiment outputs 0*; otherwise it outputs* 1 *iff* $(o_n, \gamma_n) \neq (o_{n'}, \gamma_{n'})$. $\mathsf{Adv}^{\mathrm{inv}}_{\mathcal{A}}(\kappa) = \Pr[\mathsf{Exp}^{\mathrm{inv}}_{\mathcal{A}}(\kappa) = 1]$.

Returning 0 when a node has not answered keeps the definition a safety property, in line with Remark 1: a node still preparing at line 9 defers and is not counted as disagreeing.

Lemma 2 (Pre-execution agreement). *Under Assumptions 1 and 2, two honest nodes running Algorithm 2 on the same* (d, g, b, i) *take the same branch at every line before line 14, resolve the same version* $v = v^*(a, b)$*, and compute the same* $G_{\mathrm{req}} = G(a, v, x, i, b)$*. In particular, if either node returns an ABI, missing-version or out-of-gas error, both return the same error with* $\gamma = 0$*.*

Proof. Parse is a fixed function of d, so both nodes obtain the same (a, x) or the same ABI error. Theorem 1 gives $v^*_n(a, b) = v^*_{n'}(a, b)$, so the missing-version test agrees. If a version is resolved, both nodes read $\mu_{a,v}$ from $\mathsf{Reg}(\mathcal{L}_n[1..b]) = \mathsf{Reg}(\mathcal{L}_{n'}[1..b])$ (Assumption 1 with Definition 4), hence the same $g^{\mathrm{reg}}_{a,v}$ and ABI descriptors. They also use the pricing parameters in the gas component of $\mu_{a,v}$, together with the call index i for the common transaction prefix. Thus $|x|$, i, and b determine the same branch and charge under Definition 3 and Eq. (3); comparison with the common budget g yields the same out-of-gas decision. None of these steps reads node-local state other than the prepared set, which only decides whether line 9 must prepare first, not which branch is taken. □

Theorem 4 (Invocation consistency). *Under Assumptions 1–4, for every PPT adversary* $\mathcal{A}$ *there exists a PPT* $\mathcal{B}$ *as in Theorem 2 such that*

$$\mathsf{Adv}^{\mathrm{inv}}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B}). \tag{7}$$

Moreover, whenever both nodes answer, they charge the same gas unconditionally: $\gamma_n = \gamma_{n'} = G(a, v^*(a, b), x, i, b)$ *if line 14 is reached and* $\gamma_n = \gamma_{n'} = 0$ *otherwise.*

Proof. Let W be the event that $\mathsf{Exp}^{\mathrm{inv}}_{\mathcal{A}}(\kappa)$ outputs 1, so both nodes answered and $(o_n, \gamma_n) \neq (o_{n'}, \gamma_{n'})$. Let $v = v^*(a, b)$, and let $\widehat{C}_n, \widehat{C}_{n'}$ be the byte sequences the nodes execute at line 14 when they reach it. Then $W \subseteq F_1 \cup F_2 \cup F_3$ with

$$\begin{aligned} F_1 &: \text{the nodes take different branches before line 14,} \\ F_2 &: \text{both reach line 14 and } \widehat{C}_n \neq C_{a,v} \ \vee \ \widehat{C}_{n'} \neq C_{a,v}, \\ F_3 &: \widehat{C}_n = \widehat{C}_{n'} = C_{a,v} \ \wedge \ o_n \neq o_{n'}, \end{aligned}$$

since if none occurs, both nodes either return the same pre-execution error with $\gamma = 0$, or reach line 14 with the same G_{req}, execute the same bytecode and return the same o. $\Pr[F_1] = 0$ by Lemma 2. F_2 states that some honest node executes an invocation of a at height b with bytecode other than $C_{a,v^*(a,b)}$, which is exactly the winning event of $\mathsf{Exp}^{\mathrm{bind}}_{\mathcal{A}}(\kappa)$; hence $\Pr[F_2] \leq \mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A}}(\kappa) \leq$

$\mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B})$ by Theorem 3. The bound covers both nodes at once because $\mathsf{Exp}^{\mathrm{bind}}$ already quantifies over all honest nodes. $\Pr[F_3] = 0$: on F_3 both nodes evaluate $\mathsf{Exec}(C_{a,v}, x)$ on the same arguments, and by Assumption 4 its output, including any runtime error, is a function of $(C_{a,v}, x)$ and the fixed host interface; ABI encoding of that output under the common $\tau^{\mathrm{out}}_{a,v}$ is deterministic, so $o_n = o_{n'}$. A union bound gives (7). The gas statement is Lemma 2: γ equals G_{req} when line 14 is reached and 0 otherwise, and both quantities agree without any computational assumption. □

The bound in (7) coincides with that of Theorem 3: adding Assumption 4 introduces no further probabilistic loss. The next theorem establishes that an adversary cannot evade the charge prescribed by Definition 3.

Theorem 5 (Gas-charge soundness). *Under Assumptions 1 and 2, every adversary $\mathcal{A}$, including a computationally unbounded one, has $\mathsf{Adv}^{\mathrm{gas}}_{\mathcal{A}}(\kappa) = 0$.*

Proof. Fix an invocation identified by $\mathcal{A}$ for which G^* is defined. By Assumption 1, node n uses the canonical registry and the gas component of the proposal for a. Assumption 2(iii) fixes the admitted metadata, so Theorem 1 gives the canonical version $v^*(a, b)$ and its registered charge. The encoded input x and the preceding charged calls determine $|x|$ and i by Definition 3. Hence Definition 3 selects the canonical pricing branch: either the immutable registered charge or Eq. (3) with the parameters in that gas component. In both cases, $G_{\mathrm{req}} = G^*$. Algorithm 2 returns an out-of-gas error when $g < G_{\mathrm{req}}$ and charges G_{req} before execution otherwise. Reaching line 14 therefore implies $g \geq G^*$ and $\gamma = G^*$, excluding both winning events in Definition 8. Thus $\mathsf{Exp}^{\mathrm{gas}}_{\mathcal{A}}(\kappa)$ never outputs 1. □

5 Evaluation

This evaluation asks whether EvoCrypt makes upgradeable cryptographic algorithms practical for contract-facing use. We measure the cost of making an algorithm callable and the steady-state invocation cost, comparing the coprocessor path with Solidity and Precompiled Contract baselines where appropriate.

5.1 Implementation

Experiments used a Geth-based private Clique network running modified Geth v1.17.5-unstable on a 16-core, 32 GB Aliyun server with Ubuntu 24.04 LTS, built with Go 1.25.7. Solidity contracts were compiled with solc 0.8.26, and WASM modules were built with TinyGo 0.42.0.

Table 1 gives the artifact sizes of the eight algorithms. Precompile (B) is the reference source of the client-integrated Precompiled Contract; WASM (B) is the uploaded module; Solidity (B) is the deployed runtime bytecode. Sizes and upgrade gas were remeasured on a fresh single-node Clique chain with Go 1.25.0, TinyGo 0.42.0, and solc 0.8.26 (Paris target, optimizer, 200 via-IR runs) on an Intel Core i7-10700 host running Ubuntu 22.04.5 LTS. Setup latency retains the original run.

Table 1. Source and compiled artifact sizes of the evaluated algorithms.

Algorithm	Precompile (B)	WASM(B)	Solidity(B)	Role
Add	623	6,945	171	Non-cryptographic
Sha256	3,218	8,339	358	Cryptographic hash
PolynomialMul	790	8,165	1,004	Algebraic operation
Blake2bSum256	4,911	8,868	3,721	Cryptographic hash
Pbkdf2Sha256	5,435	12,371	1,555	Key derivation
Dh2048Secret	1,595	46,102	2,489†	Key exchange
PedersenCommit	5,196	48,239	3,093†	Commitment scheme
SchnorrVerify	5,627	48,639	4,232	Signature verification

†Solidity runtime bytecode includes the inherited modular-arithmetic implementation.

5.2 Upgrade Efficiency

Figure 3 compares the cost of making an algorithm callable through EvoCrypt with deploying the equivalent Solidity contract. Receipt gas is `gasUsed` for the first upload or deployment, including intrinsic and calldata costs. The Precompiled Contract path needs no separate upload.

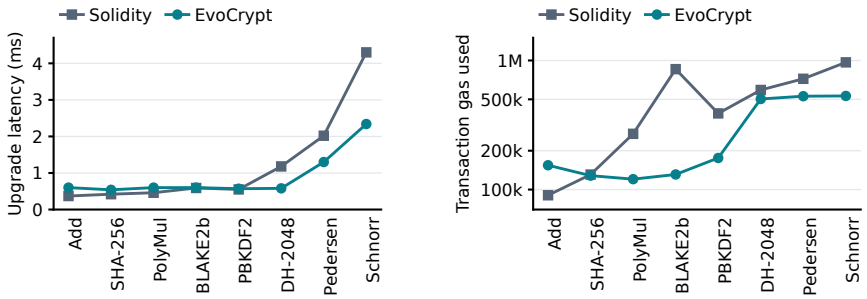


Fig. 3. Upgrade efficiency. Left: mean setup latency. Right: receipt gas for EvoCrypt upload and Solidity deployment.

The two paths were similar for lightweight algorithms and diverged as the algorithm grew heavier. Add required 0.60 ms through EvoCrypt and 0.37 ms through Solidity, whereas SchnorrVerify required 2.34 ms and 4.30 ms. Upload gas was lower for seven of the eight algorithms, by 1.9–84.7%. PBKDF2 required 175,668 gas for upload versus 388,984 gas for Solidity deployment, whereas Add remained more expensive at 154,268 versus 90,309 gas. A heavier algorithm can therefore be made callable without deploying a full Solidity implementation, at a lower setup cost than contract deployment. Lightweight cases stay comparable.

5.3 Upgrade Consistency

Figure 4 shows the upgrade path on the 20-node private Clique network. The upgrade transaction records version metadata on chain, each node prepares the WASM module locally, and invocations at or after the Activation Block resolve to the same active version. Geth-side handling finished within 0.54–2.34 ms, after which every evaluated node exposed the same callable version. Local readiness affects only preparation timing; version selection is fixed by the shared metadata and block height. The experiment is a controlled private-network validation and leaves wide-area propagation and adversarial delay unmeasured.

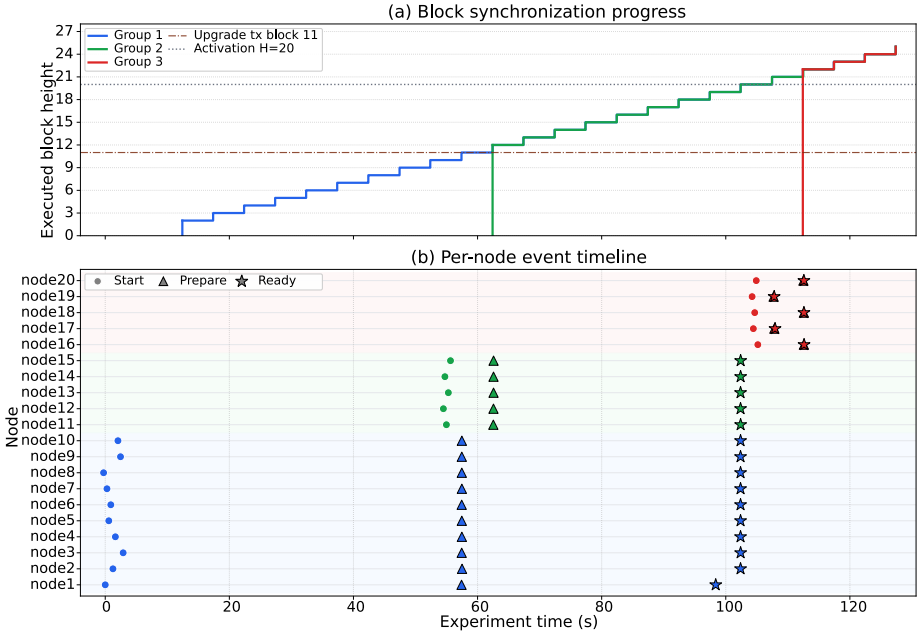


Fig. 4. Upgrade consistency across nodes: synchronization, preparation, and readiness.

5.4 Execution Efficiency

Steady-state invocation was measured on a single node. PBKDF2 used HMAC-SHA-256 to derive a 32-byte key, and the iteration count varied from 1 to 100 across Solidity, the Precompiled Contract, and EvoCrypt (Figure 5, left). Each point is the median `eth_call` latency of thirty measured calls after warm-up, including RPC overhead and excluding compilation, upload, and deployment. EvoCrypt stayed close to Solidity when each call did little work: at one iteration

the times were 0.817 ms and 0.744 ms. At 100 iterations they were 2.967 ms and 8.152 ms, a 63.6% reduction. The Precompiled Contract path remained fastest, at 0.639 ms. The latency advantage therefore grows with the work per call.

Calling gas fixed ten internal iterations and varied the number of calls inside one transaction. The baseline used $g_0 = 3000$, $\alpha = 1/2$, and $N_0 = 2$ in Eq. (3); Figure 5 (right) also shows the variants $g_0 = 6000$, $\alpha = 3/4$, and $N_0 = 4$. The curves are median receipt `gasUsed`. Under the baseline, EvoCrypt used less gas than Solidity at all nine call counts. One call cost 34,367 gas versus 297,297, and sixteen calls cost 2,703,489 versus 4,333,408. The saving narrowed from 88.4% to 37.6% as the tariff charged repeated calls more heavily. Raising g_0 or α increased that charge, and raising N_0 postponed the increase. The gas advantage holds for this schedule and narrows as calls accumulate.

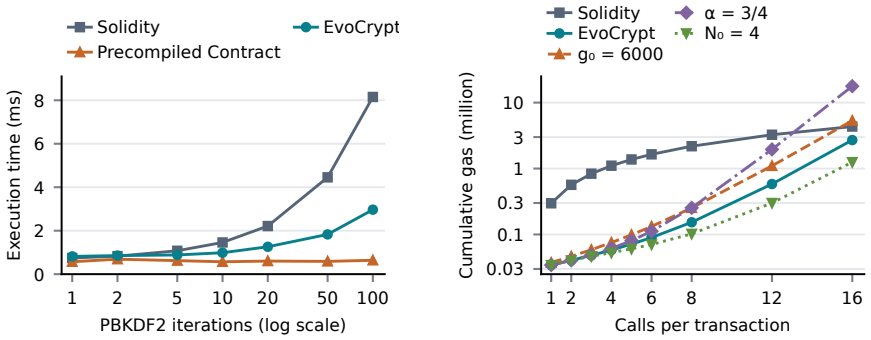


Fig. 5. PBKDF2 efficiency. Left: invocation latency versus iterations. Right: cumulative gas versus calls per transaction.

5.5 Functional Validation

Table 2 reports standalone verification of post-quantum signatures and Groth16 proofs. These algorithms are more demanding than the contract-facing suite, lie beyond practical direct Solidity execution in this setting, and are absent from the evaluated Precompiled Contract catalog. Each one was uploaded, prepared, and invoked through the same interface, without a client modification. The times exclude RPC, upload, and confirmation, and are separate from the invocation measurements in Section 5.4.

Table 2. Mean standalone verification time for computationally intensive cryptographic algorithms. Ratios are WASM/Precompiled Contract.

Algorithm	WASM (ms)	Precompile (ms)	Ratio
Aigis-Sig2Verify	0.226	0.106	2.13
Dilithium3Verify	0.282	0.141	2.00
ML-DSA-65Verify	0.286	0.141	2.03
SLH-DSA-SHAKE-192fVerify	7.585	3.964	1.91
Groth16Verify/BLS12-381	19.809	3.518	5.63
Groth16Verify/BN254	11.676	2.182	5.35

All six verifiers executed successfully. A contract can therefore call algorithms that are not in the client catalog, and a later upgrade can replace the implementation through the same interface. WASM time was 1.91–5.63 times the corresponding Precompiled Contract time, with the largest ratio for Groth16Verify/BLS12-381. EvoCrypt extends the set of callable algorithms; where a Precompiled Contract already exists, that path remains faster.

6 Conclusion

EvoCrypt addresses the problem of evolving contract-facing cryptographic capabilities without embedding complex algorithms in EVM bytecode or modifying the client for every update. Its design separates the stable contract-facing interface from replaceable client-side WASM implementations, while deterministic version selection and algorithm-level accounting preserve consistent invocation rules.

Future work will focus on two directions. First, we will develop a more accurate gas-pricing model that reflects the computational and memory costs of cryptographic operations while retaining deterministic charges across nodes. Second, we will investigate execution and scheduling strategies to reduce the transaction-validation latency introduced by complex algorithms in the coprocessor. The aim is to limit delays to block production while preserving deterministic execution and agreement across nodes.

References

1. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. https://bitcoin.org/bitcoin.pdf (2008), accessed 21 August 2026
2. Zou, W., Lo, D., Kochhar, P.S., Le, X.B.D., Xia, X., Feng, Y., Chen, Z., Xu, B.: Smart contract development: Challenges and opportunities. IEEE Transactions on Software Engineering **47**(10), 2084–2106 (2021)
3. Wood, G.: Ethereum: A secure decentralised generalised transaction ledger. https://ethereum.github.io/yellowpaper/paper.pdf (2014), ethereum Yellow Paper. Accessed 21 August 2026

4. Ethereum.org: Ethereum virtual machine (EVM). https://ethereum.org/developers/docs/evm/, accessed 21 August 2026
5. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM Review **41**(2), 303–332 (1999)
6. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the 28th Annual ACM Symposium on Theory of Computing. pp. 212–219. Association for Computing Machinery, New York, NY, USA (1996)
7. National Institute of Standards and Technology: Post-quantum cryptography. https://csrc.nist.gov/projects/post-quantum-cryptography (2026), updated 5 August 2026. Accessed 21 August 2026
8. EVM Codes: Precompiled contracts. https://www.evm.codes/precompiled, accessed 21 August 2026
9. Input Output: Combinator makes easy work of hard forks. https://iohk.io/en/blog/posts/2020/07/23/combinator-makes-easy-work-of-hard-forks/ (2020), accessed 15 September 2026
10. Cardano Foundation: About hard forks. https://docs.cardano.org/about-cardano/evolution/about-hard-forks, accessed 15 September 2026
11. Ciampi, M., Karayannidis, N., Kiayias, A., Zindros, D.: Updatable blockchains. In: Chen, L., Li, N., Liang, K., Schneider, S.A. (eds.) 2020 European Symposium on Research in Computer Security. Lecture Notes in Computer Science, vol. 12309, pp. 590–609. Springer, Cham (2020)
12. Zindros, D.: Soft power: Upgrading chain macroeconomic policy through soft forks. In: Bernhard, M., Bracciali, A., Gudgeon, L., Haines, T., Klages-Mundt, A., Matsuo, S., Perez, D., Sala, M., Werner, S. (eds.) 2021 Financial Cryptography and Data Security International Workshops. Lecture Notes in Computer Science, vol. 12676, pp. 467–481. Springer, Berlin, Heidelberg (2021)
13. Wang, C., Li, P., Fan, X., Tang, Z., Zeng, Y., Sakurai, K.: Phoenix: A live upgradable blockchain client. IEEE Transactions on Sustainable Computing **8**(4), 703–714 (2023)
14. Polkadot: Runtime upgrades. https://docs.polkadot.com/parachains/maintain/runtime-upgrades/, accessed 15 September 2026
15. Rodler, M., Li, W., Karame, G.O., Davi, L.: EVMPatch: Timely and automated patching of ethereum smart contracts. In: Proceedings of the 30th USENIX Security Symposium. pp. 1289–1306. USENIX Association, Berkeley, CA, USA (2021)
16. Kumar, G., Saha, R., Conti, M., Buchanan, W.J.: LEAGAN: A decentralized version-control framework for upgradeable smart contracts. IEEE Transactions on Services Computing **18**(3), 1529–1542 (2025)
17. Reitwiessner, C.: EIP-196: Precompiled contracts for addition and scalar multiplication on the elliptic curve alt_bn128. Ethereum Improvement Proposals, no. 196. https://eips.ethereum.org/EIPS/eip-196 (2017), final. Accessed 21 August 2026
18. Buterin, V., Reitwiessner, C.: EIP-197: Precompiled contracts for optimal ate pairing check on the elliptic curve alt_bn128. Ethereum Improvement Proposals, no. 197. https://eips.ethereum.org/EIPS/eip-197 (2017), final. Accessed 21 August 2026
19. Zhang, Y., Crowcroft, J., Li, D., Zhang, C., Li, H., Wang, Y., Yu, K., Xiong, Y., Chen, G.: KylinX: A dynamic library operating system for simplified and efficient cloud virtualization. In: Proceedings of the 2018 USENIX Annual Technical Conference. pp. 173–186. USENIX Association, Berkeley, CA, USA (2018)
20. Pina, L., Andronidis, A., Hicks, M., Cadar, C.: MVEDSUA: Higher availability dynamic software updates via multi-version execution. In: Proceedings of the 24th

International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 573–585. Association for Computing Machinery, New York, NY, USA (2019)
21. Huang, H., Xiang, C., Zhong, L., Zhou, Y.: PYLIVE: On-the-fly code change for python-based online services. In: Proceedings of the 2021 USENIX Annual Technical Conference. pp. 349–363. USENIX Association, Berkeley, CA, USA (2021)
22. Ajmani, S., Liskov, B., Shrira, L.: Modular software upgrades for distributed systems. In: Thomas, D. (ed.) 2006 European Conference on Object-Oriented Programming. Lecture Notes in Computer Science, vol. 4067, pp. 452–476. Springer, Berlin, Heidelberg (2006)
23. Baresi, L., Ghezzi, C., Ma, X., Panzica La Manna, V.: Efficient dynamic updates of distributed components through version consistency. IEEE Transactions on Software Engineering **43**(4), 340–358 (2017)
24. World Wide Web Consortium: WebAssembly Core Specification. https://www.w3.org/TR/wasm-core-2/, w3C Candidate Recommendation Draft. Accessed 3 September 2026
25. Garay, J.A., Kiayias, A., Leonardos, N.: The bitcoin backbone protocol: Analysis and applications. In: Oswald, E., Fischlin, M. (eds.) 2015 Annual International Conference on the Theory and Applications of Cryptographic Techniques, Part II. Lecture Notes in Computer Science, vol. 9057, pp. 281–310. Springer, Berlin, Heidelberg (2015)
26. Rogaway, P.: Formalizing human ignorance: Collision-resistant hashing without the keys. In: Nguyen, P.Q. (ed.) 2006 International Conference on Cryptology in Vietnam. Lecture Notes in Computer Science, vol. 4341, pp. 211–228. Springer, Berlin, Heidelberg (2006)